

SIMATS ENGINEERING

ASSESSMENT TOOL 1: Expert System Development Project

COURSE NAME: ARTIFICIAL INTELLIGENCE COURSE CODE: MLA01
AND EXPERT SYSTEMS

COURSE OUTCOME COVERED

CO 5: Construct rule-based expert systems using production rules, forward and backward chaining, and by distinguishing procedural and non-procedural paradigms across development stages.(BTL 3)

Assessment Tool Weightage: 40%

Total Marks: 100

Time: 90 Mins

No. of Questions: 1

Annexure A

Expert System Development Project

Code of Conduct:

I B.Jahnavi/192325043 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: b.jahnavi

Criterion	Weightage	Marks Obtained
1. Problem Analysis and Domain Understanding	10%	
2. Knowledge Acquisition and Knowledge Representation	10%	
3. Production Rule / Knowledge Base Design	15%	
4. Prolog Implementation and Programming	15%	
5. Forward Chaining Implementation and Demonstration	10%	
6. Backward Chaining Implementation and Demonstration	10%	
7. Procedural and Non-Procedural Paradigm Analysis	10%	
8. Unification, Backtracking and Inference Accuracy	10%	
9. Testing, Results and System Demonstration	5%	
10. Documentation, GitHub Repository and Technical Presentation	5%	
Total	100%	

Signature of the Faculty

Total Marks Awarded

ANSWER ANY ONE OF THE FOLLOWING

Tool: SWI-Prolog

Usage of Prolog: Students shall use **Prolog** to develop a rule-based expert system by representing domain knowledge as **facts and production rules**, and implementing reasoning through **forward and backward chaining**. The project should demonstrate Prolog's **declarative/non-procedural nature**, logical inference, rule matching, and automated conclusion generation.

S.No.	Scenario-Based Expert System Development Question
1	Medical Diagnosis Expert System: Develop a rule-based expert system that identifies possible diseases based on symptoms, patient observations, and basic clinical information. Represent domain knowledge using production rules and implement both forward and backward chaining to derive conclusions.
2	Crop Disease Advisory System: Develop an expert system that identifies possible crop diseases based on leaf symptoms, soil conditions, weather conditions, and visible plant characteristics. Construct suitable production rules and demonstrate diagnosis using forward and backward chaining.
3	Automobile Fault Diagnosis System: Design an expert system that identifies probable vehicle faults based on symptoms such as engine noise, starting problems, warning indicators, abnormal vibration, and reduced mileage. Implement the knowledge base using production rules and compare forward and backward reasoning.
4	Student Academic Advisory System: Develop an expert system that recommends suitable academic interventions based on attendance, internal marks, assignment performance, learning difficulties, and previous academic performance. Use production rules and demonstrate both forward and backward chaining.
5	Cybersecurity Threat Diagnosis System: Construct a rule-based expert system that identifies possible cybersecurity threats from events such as repeated login failures, unusual network traffic, suspicious file activity, and unauthorized access attempts. Implement production rules and demonstrate how forward and backward chaining arrive at a threat classification.

Report Format:

Stage	Student Activity	Expected Output
1. Domain Analysis	Identify the problem domain, entities, facts, symptoms/conditions, and possible conclusions.	Problem definition and domain model
2. Knowledge Acquisition	Collect domain knowledge from reliable sources or domain experts.	Knowledge specification
3. Knowledge Representation	Convert domain knowledge into IF–THEN production rules.	Knowledge base
4. Inference Design	Implement forward chaining and backward chaining.	Inference engine
5. Paradigm Analysis	Distinguish how the solution can be expressed using procedural and non-procedural approaches.	Paradigm comparison
6. System Development	Integrate the knowledge base, inference engine, user interface, and explanation facility.	Working expert system
7. Testing	Test the system using multiple facts and scenarios.	Test cases and outputs
8. Evaluation	Analyze correctness, coverage, consistency, and reasoning performance.	Evaluation results
9. Documentation	Prepare technical documentation and GitHub repository.	Project report + GitHub
10. Demonstration	Demonstrate the system and explain the reasoning process.	Project presentation/viva

EVALUATION RUBRICS:

Criterion	Weightage (%)	Excellent	Good	Average	Poor
1. Problem Analysis and Domain Understanding	10%	9–10% – Clearly analyzes the real-world problem, domain entities, facts, conditions, goals, and expected conclusions.	7–8% – Good understanding of the domain with minor omissions.	5–6% – Basic domain understanding with several missing elements.	0–4% – Poor or incorrect understanding of the problem domain.
2. Knowledge Acquisition and Representation	10%	9–10% – Acquires relevant domain knowledge and represents it accurately using well-structured Prolog facts and rules.	7–8% – Knowledge is appropriately represented with minor gaps.	5–6% – Basic knowledge representation with missing or unclear facts/rules.	0–4% – Inadequate or inappropriate knowledge representation.
3. Production Rule / Knowledge Base Design	15%	13–15% – Develops a comprehensive, consistent, and logically structured production-rule knowledge base with appropriate facts and rules.	10–12% – Well-designed knowledge base with minor inconsistencies or omissions.	7–9% – Basic rule base developed but several rules are incomplete or unclear.	0–6% – Knowledge base is incomplete, inconsistent, or inappropriate.
4. Prolog Implementation	15%	13–15% – Implements a fully functional expert system using Prolog facts, predicates, rules, variables, unification, and backtracking effectively.	10–12% – Functional Prolog implementation with minor technical issues.	7–9% – Basic implementation works with limited use of Prolog features.	0–6% – Implementation is incomplete, incorrect, or non-functional.

5. Forward Chaining	10%	9–10% – Correctly implements and demonstrates forward chaining from initial facts through applicable rules to final conclusions.	7–8% – Forward chaining works correctly with minor limitations.	5–6% – Basic forward reasoning demonstrated but with incomplete rule processing.	0–4% – Forward chaining is missing or incorrectly implemented.
6. Backward Chaining	10%	9–10% – Effectively demonstrates goal-driven backward chaining using Prolog's inference mechanism and clearly traces the reasoning process.	7–8% – Backward chaining works correctly with minor gaps.	5–6% – Basic backward reasoning demonstrated with limited explanation.	0–4% – Backward chaining is missing or incorrectly implemented.
7. Procedural and Non-Procedural Paradigm Analysis	10%	9–10% – Clearly distinguishes procedural and declarative/non-procedural approaches and demonstrates their application in the Prolog system.	7–8% – Provides a good distinction with minor conceptual gaps.	5–6% – Basic distinction is explained but lacks practical demonstration.	0–4% – Paradigms are misunderstood or not addressed.
8. Inference, Unification and Reasoning Accuracy	10%	9–10% – Produces accurate conclusions using unification, backtracking, and logical inference across diverse test cases.	7–8% – Reasoning is mostly accurate with minor errors.	5–6% – Basic inference works but produces occasional incorrect results.	0–4% – Inference is unreliable or incorrect.

9. Testing, Results and System Demonstration	5%	5% – Provides comprehensive test cases, accurate outputs, reasoning traces, and a convincing live demonstration.	4% – Good testing and demonstration with minor gaps.	2–3% – Limited test cases and basic demonstration.	0–1% – Testing or demonstration is missing/inadequate.
10. Documentation, GitHub and Technical Presentation	5%	5% – Complete documentation, well-organized GitHub repository, clear code, references, and professional presentation.	4% – Good documentation and repository with minor issues.	2–3% – Basic documentation/repository with several omissions.	0–1% – Poor documentation, missing repository, or inadequate presentation.
Total	100%				

Automobile Fault Diagnosis Expert System

1. Problem Analysis and Domain Understanding

Most car owners do not know why their vehicle is behaving strangely. When a car makes a strange noise or refuses to start, a normal person cannot tell what part of the car is causing the problem. Only an experienced mechanic can look at the symptoms and guess the fault correctly. This project tries to copy that thinking process using Prolog.

The goal of this system is simple: the user tells the system what problems the car is showing, and the system tells back the most likely fault. This is exactly how a mechanic works — he listens to the symptoms first and then gives his diagnosis based on experience.

Domain entities

- Symptoms – engine noise, starting trouble, warning lights, vibration, low mileage, braking noise, etc.
- Faults – the actual problem in the car, like dead battery, faulty alternator, worn spark plugs, low engine oil, worn brake pads, wheel alignment problem, faulty oxygen sensor.
- Facts – symptom(SymptomName) stored as Prolog facts.
- Rules – IF a group of symptoms is true THEN a fault is concluded.
- Goal – to reach the final diagnosis (the fault) using the symptoms available.

2. Knowledge Acquisition

For this project the knowledge was collected from common automobile service manuals, YouTube videos of car mechanics explaining basic diagnosis, and general knowledge shared by two local mechanics who were asked informally about common car problems. Only the most common and well known faults were picked so that the rules stay simple and reliable.

The knowledge collected can be listed as symptom–fault pairs, shown below:

Fault	Typical Symptoms
Dead battery	engine won't start, clicking sound, dim dashboard lights
Faulty alternator	battery warning light, dim lights, engine stalls while driving
Worn spark plugs	engine misfires, rough idling, reduced mileage
Clogged fuel filter	engine hesitates, reduced mileage, difficulty starting
Low engine oil	knocking noise, oil warning light, engine overheating
Worn brake pads	squealing while braking, vibration while braking
Wheel alignment issue	abnormal vibration, uneven tyre wear, car pulls to one side
Faulty oxygen sensor	check engine light, reduced mileage, rough idling

3. Knowledge Representation – Production Rules

The knowledge shown above was converted into IF-THEN production rules using Prolog syntax. In Prolog a rule is written as Head :- Body, which means "Head is true IF Body is true". Each fault is the head, and the group of symptoms needed is the body.

Below are a few sample production rules from the knowledge base (full code is on GitHub):

```
% IF engine_wont_start AND clicking_sound AND dashboard_lights_dim
% THEN fault is dead_battery
fault(dead_battery) :-
    symptom(engine_wont_start),
    symptom(clicking_sound),
    symptom(dashboard_lights_dim).
```

```
% IF engine_misfires AND rough_idling AND reduced_mileage
% THEN fault is worn_spark_plugs
fault(worn_spark_plugs) :-
    symptom(engine_misfires),
    symptom(rough_idling),
    symptom(reduced_mileage).
```

Notice that these rules do not say HOW to search for the answer. They just state the relationship between symptoms and faults. This is the main idea behind knowledge representation in Prolog — the knowledge is written down and Prolog's own inbuilt engine decides how to use it.

4. Complete Prolog Implementation

The full knowledge base with all facts, rules and the two chaining mechanisms is given below.

```
% =====
% AUTOMOBILE FAULT DIAGNOSIS EXPERT SYSTEM
% =====

:- dynamic(symptom/1).
:- dynamic(diagnosed/1).

% ----- SAMPLE SYMPTOM FACTS -----
% (these get asserted/retracted for each test case)
symptom(engine_wont_start).
symptom(clicking_sound).
symptom(dashboard_lights_dim).
```


% ----- PRODUCTION RULES (used for backward chaining) -----

fault(dead_battery) :-

 symptom(engine_wont_start),
 symptom(clicking_sound),
 symptom(dashboard_lights_dim).

fault(faulty_alternator) :-

 symptom(battery_warning_light),
 symptom(dashboard_lights_dim),
 symptom(engine_stalls_while_driving).

fault(worn_spark_plugs) :-

 symptom(engine_misfires),
 symptom(rough_idling),
 symptom(reduced_mileage).

fault(clogged_fuel_filter) :-

 symptom(engine_hesitates),
 symptom(reduced_mileage),
 symptom(difficulty_starting).

fault(low_engine_oil) :-

 symptom(knocking_noise),
 symptom(oil_warning_light),
 symptom(engine_overheating).

fault(worn_brake_pads) :-

 symptom(squealing_while_braking),
 symptom(vibration_while_braking).

fault(wheel_alignment_issue) :-

 symptom(abnormal_vibration),
 symptom(uneven_tyre_wear),
 symptom(car_pulls_one_side).

fault(faulty_oxygen_sensor) :-

 symptom(check_engine_light),
 symptom(reduced_mileage),
 symptom(rough_idling).

```
% ----- SAME RULES WRITTEN AS FACTS (used for forward chaining) -----
fault_rule(dead_battery,
    [engine_wont_start,          clicking_sound,          dashboard_lights_dim]).
fault_rule(faulty_alternator,
    [battery_warning_light,      dashboard_lights_dim,      engine_stalls_while_driving]).
fault_rule(worn_spark_plugs,
    [engine_misfires,           rough_idling,           reduced_mileage]).
fault_rule(clogged_fuel_filter,
    [engine_hesitates,          reduced_mileage,          difficulty_starting]).
fault_rule(low_engine_oil,
    [knocking_noise,           oil_warning_light,       engine_overheating]).
fault_rule(worn_brake_pads,
    [squealing_while_braking, vibration_while_braking]).
fault_rule(wheel_alignment_issue,
    [abnormal_vibration,        uneven_tyre_wear,         car_pulls_one_side]).
fault_rule(faulty_oxygen_sensor,
    [check_engine_light, reduced_mileage, rough_idling]).
```

```
% ----- FORWARD CHAINING ENGINE -----
```

```
all_present([]).
```

```
all_present([H|T]) :-
```

```
    symptom(H),
```

```
    all_present(T).
```

```
forward_chaining      :-
```

```
    fault_rule(Fault, Symptoms),
```

```
    all_present(Symptoms),
```

```
    \+      diagnosed(Fault),
```

```
    assertz(diagnosed(Fault)),
```

```
    format("Forward chaining fired -> ~w~n", [Fault]),
```

```
    fail.
```

```
forward_chaining :-
```

```
    findall(F, diagnosed(F), Result),
```

```
    ( Result = []
```

```
    -> writeln('No fault matched the given symptoms.')
```

```
    ; format("Final diagnosis (forward chaining): ~w~n", [Result])
```

```
    ).
```

```
% ----- BACKWARD CHAINING -----
```

```
backward_chaining(Fault) :-
```

```
    fault(Fault),
```

```
format("Backward chaining proved fault: ~w~n", [Fault]).
```

```
% ----- HELPER TO RESET SYMPTOMS FOR A NEW CASE -----
```

```
reset_case      :-  
    retractall(symptom(_)),  
    retractall(diagnosed(_)).
```

The dynamic declaration at the top allows facts about symptom/1 and diagnosed/1 to be added or removed while the program is running. This is needed because for every new car (new test case) the old symptoms have to be cleared first using reset_case, and then the new symptoms are asserted.

5. Forward Chaining Implementation and Demonstration

Forward chaining starts from the facts (the symptoms we already know) and moves forward, applying rules one by one, until no more new fact can be produced. This is a data-driven method — we begin with data and keep matching rules against it.

Example run for a car that will not start:

```
?-                                reset_case,  
    assertz(symptom(engine_wont_start)),  
    assertz(symptom(clicking_sound)),  
    assertz(symptom(dashboard_lights_dim)),  
    forward_chaining.
```

Forward chaining fired -> dead_battery

Final diagnosis (forward chaining): [dead_battery]

true.

Here Prolog checked every fault_rule one at a time, and since all three symptoms needed for dead_battery were present, that rule fired and the fault got added using assertz. The engine kept scanning the rest of the rules too, but since no other rule matched, only one fault came out at the end.

6. Backward Chaining Implementation and Demonstration

Backward chaining works the opposite way. It starts from a goal (a fault we want to check) and works backward to see if the needed symptoms are true. This is a goal-driven method, and it is actually how Prolog naturally answers any query, so this part of the program uses Prolog's own built-in resolution.

Example run, checking if the fault is worn_spark_plugs:

```
?-                                reset_case,  
    assertz(symptom(engine_misfires)),  
    assertz(symptom(rough_idling)),  
    assertz(symptom(reduced_mileage)),
```

```
backward_chaining(worn_spark_plugs).
```

Backward chaining proved fault: worn_spark_plugs true.

Prolog took the goal fault(worn_spark_plugs), looked at its rule, and then tried to prove each symptom in the body one by one from left to right. Since all three symptom facts were already asserted, each one matched immediately (this matching step is called unification) and the goal succeeded. If we ask for a fault that does not match, Prolog will simply reply false, because it could not prove the goal.

7. Procedural vs Non-Procedural Paradigm Analysis

A procedural language like C or Python needs the programmer to write down every step of how to solve the problem. For example, the same dead-battery check written in a procedural style would look like this:

```
def check_dead_battery(symptoms):  
    if ('engine_wont_start' in symptoms and  
        'clicking_sound' in symptoms and  
        'dashboard_lights_dim' in symptoms):  
        return 'dead_battery'  
    return None
```

In this style, the programmer has to manually write the loop, the condition checking, and the order in which things are tested. If a new fault is added, the whole function structure has to be changed.

Prolog is different. It is a non-procedural (declarative) language. We only describe WHAT is true, not HOW to find it:

```
fault(dead_battery) :-  
    symptom(engine_wont_start),  
    symptom(clicking_sound),  
    symptom(dashboard_lights_dim).
```

There is no loop, no if-else, and no explicit search order written by us. Prolog's own inference engine automatically searches through the facts, tries to unify variables, and backtracks if a path fails. This is the biggest advantage of using Prolog for an expert system — adding a new fault just means adding one new rule, without touching any existing code.

Point	Procedural (e.g. Python/C)	Non-Procedural (Prolog)
Control flow	Written by the programmer (loops, if-else)	Handled automatically by the inference engine
Focus	How to solve it, step by step	What is true, as facts and rules
Adding new knowledge	May need to edit existing code	Just add a new fact or rule

Searching for an answer	Manual	Automatic, using unification and backtracking
-------------------------	--------	---

8. Unification, Backtracking and Inference Accuracy

Unification is the process Prolog uses to match a query with a fact or a rule head. When we ask `backward_chaining(worn_spark_plugs)`, Prolog tries to unify `worn_spark_plugs` with the `Fault` variable in `fault(Fault)` and, once matched, moves to prove the body.

Backtracking happens when a chosen path fails and Prolog goes back to try another option. For instance, if we ask `fault(X)` with no bindings, Prolog will first try `dead_battery`. If the required symptoms are not all present, that attempt fails, and Prolog automatically backtracks to try `faulty_alternator` next, and so on, until a rule succeeds or all rules are exhausted.

A short test of this backtracking behaviour:

```
?-                reset_case,
  assertz(symptom(knocking_noise)),
  assertz(symptom(oil_warning_light)),
  assertz(symptom(engine_overheating)),
  fault(X).
```

`X = low_engine_oil.`

Here Prolog tried every fault clause one after another (backtracking through the choice points) until it found the one, `low_engine_oil`, whose body could be fully proved with the current symptom facts.

9. Testing, Results and System Demonstration

The system was tested with eight different symptom combinations, one for each fault in the knowledge base. All eight produced correct results, and one extra case was tested with an incomplete symptom set to check that the system correctly says no fault was found instead of guessing wrong.

Test No.	Symptoms Given	Expected Fault	System Output	Result
T1	engine_wont_start, clicking_sound, dashboard_lights_dim	dead_battery	dead_battery	Pass
T2	battery_warning_light, dashboard_lights_dim, engine_stalls_while_driving	faulty_alternator	faulty_alternator	Pass
T3	engine_misfires, rough_idling, reduced_mileage	worn_spark_plugs	worn_spark_plugs	Pass
T4	engine_hesitates, reduced_mileage, difficulty_starting	clogged_fuel_filter	clogged_fuel_filter	Pass
T5	knocking_noise, oil_warning_light,	low_engine_oil	low_engine_oil	Pass

	engine_overheating			
T6	squealing_while_braking, vibration_while_braking	worn_brake_pads	worn_brake_pads	Pass
T7	abnormal_vibration, uneven_tyre_wear, car_pulls_one_side	wheel_alignment_issue	wheel_alignment_issue	Pass
T8	check_engine_light, reduced_mileage, rough_idling	faulty_oxygen_sensor	faulty_oxygen_sensor	Pass
T9	reduced_mileage only (incomplete)	no confident fault	No fault matched the given symptoms.	Pass

For system demonstration, the program was run directly from the SWI-Prolog console (swipl -s automobile_expert.pl). Each test case was entered as a query, the reasoning trace (which rule fired, which symptoms matched) was shown on screen, and the final fault was printed. A short screen-recording of this run along with the console output text file is included in the GitHub repository.

10. Evaluation of the System

- Correctness – All 9 test cases gave the expected result, so the rule base is logically correct for the cases it was designed for.
- Coverage – The system currently covers 8 common automobile faults. It does not cover rare or very technical faults (like ECU chip failure), which is a limitation.
- Consistency – No two rules contradict each other because every fault has its own separate, non-overlapping set of symptoms.
- Reasoning performance – Both forward and backward chaining returned answers instantly for this rule size; performance was not an issue since the knowledge base is small.

Conclusion

This project shows that a car fault diagnosis problem, which is normally solved using a mechanic's experience, can be represented cleanly using facts and production rules in Prolog. Forward chaining was useful to show how the system reacts automatically once symptoms are known, and backward chaining was useful to check one specific fault directly. The comparison with procedural code made it clear why Prolog's declarative style is a natural fit for expert systems the knowledge stays separate from the way it is used, so the knowledge base can keep growing without breaking the existing logic.